

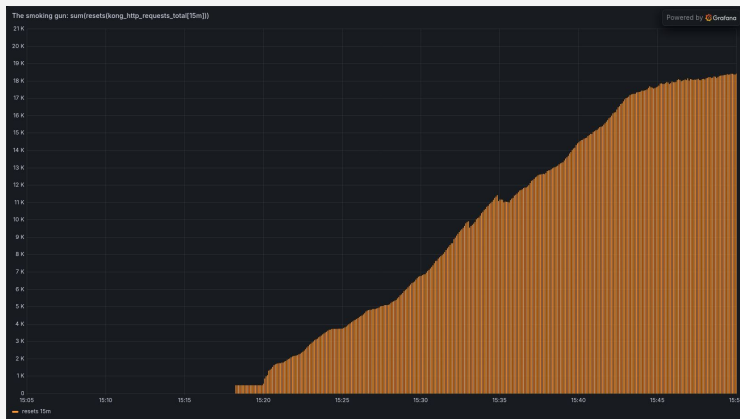
Prometheus (Didn't) Lie

A Mystery From the Field

Mehmet Baykara
Grafana Labs – PromCon 2026



The spike or not the spike?



- First check, like everyone: did the pod restart? restartCount 0, up == 1 throughout.
- But resets() fired on pods that never restarted. Repro, left: 18,425 resets in 15 min. restartCount stayed at 0.
- The restart check is a false negative. Kubernetes sees PID 1. It cannot see a counter dying inside a healthy process.



Where the counter actually lives

```
-- resty/counter.lua:37
_, err, _ = self.dict:incr(k, v, 0)
--      ^ forcible: true if it
--      evicted items via LRU
--      ...discarded

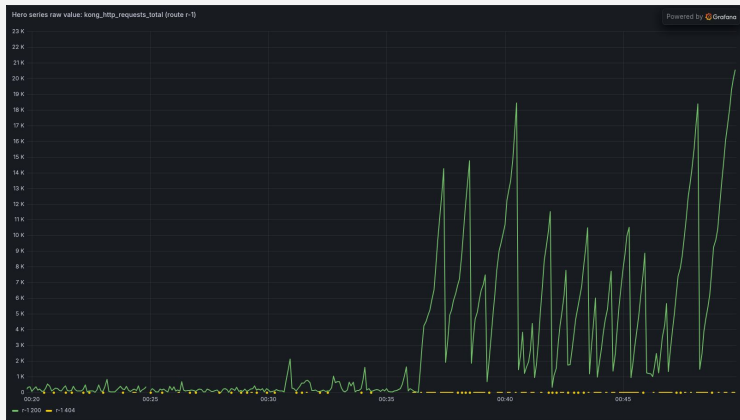
-- prometheus.lua:931
keys = self.dict:get_keys(0)
-- no record of which series
-- SHOULD exist. Survivors only.
```

- Request -> Lua table in one nginx worker. A 1s timer flushes it into ONE lua_shared_dict. Default 5m, "temporary workaround".
- Dict full: incr() LRU-evicts other live series. OpenResty returns a forcible flag. Line 37 assigns it to _.
- /metrics dumps dict:get_keys(0). An evicted series is not 0, not stale, simply absent. Next request recreates it at 0 + buffered delta.



One series, /metrics read 20 times in 10 seconds

What that looks like on the wire



- 3,372 -> 320 671 -> 177 280 -> 234
- A monotonic counter, going down three times in ten seconds.
- docker inspect: restarts=0, StartedAt unchanged.
- The sawtooth on the left is a counter inside a healthy process.



The watchdog died with the patient

```
-- upstream, fixed 2022 (#147), :37
_, err, forcible =
  self.dict:incr(k, v, 0)
if forcible then
  ngx.log(ngx.ERR, "...lru eviction")

-- Kong lua-resty-counter 0.2.1 (2020),
:37
_, err, _ = self.dict:incr(k, v, 0)

-- both: error counter in self.dict
```

- Kong wrote this counter and gave it upstream (2020). Upstream fixed it in 2022: logs "lru eviction", bumps the error counter. Kong still loads its 2020 copy.
- Either way the error counter lives in the same dict. One run: 601 errors in a wipe, one scrape, then 0.
- APISIX: a fork of the same library, overflow after 2-3 weeks (#11948). statsd_exporter TTL (#129). Python multiprocessing files



What I do differently now

```
sum by (pod) (  
    resets(requests_total[1h]))  
  
kong_memory_lua_shared_dict_bytes  
    {shared_dict="prometheus_metrics"}  
/  
kong_memory_lua_shared_dict_total_bytes  
    {shared_dict="prometheus_metrics"}  
> 0.9
```

- Before trusting any counter panel, flat or spiky: put resets() by pod next to it.
- Window $\geq 4 \times$ scrape interval:
\$__rate_interval, with the data source scrape interval set to match. Field: [1m] at a 60s scrape = one sample, 98% of series dropped. Longer windows hide eviction, they do not fix it.
- Alert on the gauge that lives OUTSIDE the dict (per-worker memory)
- Start timestamps (tested on 3.15.0) count a new series from its first sample and catch resets.



Prometheus didn't lie.

Danke!

